

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN

**ĐỀ TÀI: THIẾT KẾ RTL VÀ MÔ PHỎNG PHỤC VỤ
ĐÁNH GIÁ RÒ RỈ BẰNG TVLA**

Môn học: CE433.Q21 – Thiết kế hệ thống SoC

Giảng viên hướng dẫn: Trương Văn Cương

Thực hiện bởi nhóm 20, bao gồm:

- | | | |
|---------------------------|----------|-------------|
| 1. Nguyễn Đặng Phương Duy | 23520372 | Trưởng nhóm |
| 2. Vũ Đại Dương | 23520359 | Thành viên |

Thời gian thực hiện: 21/04/2026 – 04/05/2026

MỤC LỤC

MỤC LỤC	2
TÓM TẮT	3
Chương I. TỔNG QUAN.	4
1. Giới thiệu đề tài.	4
2. Bài toán thiết kế.	4
Chương II. THIẾT KẾ RTL VÀ MÔ PHỎNG CHỨC NĂNG.....	5
1. Kiến trúc RTL.	5
2. Phân tích chức năng.....	5
Chương III. TỔNG HỢP TÀI NGUYÊN.....	7
Chương IV. PHÂN TÍCH TVLA.....	8
1. Môi trường thu thập traces cho TVLA.	8
2. Phân tích Switching Activity.	8
3. Phân tích Switching Activity.	9
4. So sánh với kiến trúc khác.....	10
Chương V. KẾT LUẬN.....	11
PHỤ LỤC	12

TÓM TẮT

Bài tập lớn này trình bày quá trình thiết kế, tổng hợp phân cứng và đánh giá rò rỉ bảo mật kênh bên cho khối tính toán MAC Engine. Thiết kế được hiện thực hóa bằng ngôn ngữ mô tả phần cứng Verilog theo kiến trúc Pipeline, đảm bảo độ chính xác 100% về mặt chức năng và đạt hiệu năng tối ưu.

Để đánh giá mức độ an toàn của thiết kế, bài tập sử dụng phương pháp kiểm định thống kê TVLA thông qua việc phân tích dữ liệu lật bit từ 2000 mẫu mô phỏng. Kết quả cho thấy mạch tồn tại rò rỉ thông tin nghiêm trọng, đậm thủng ngưỡng an toàn ± 4.5 .

Đặc biệt, thông qua việc đối chiếu trực tiếp giữa hai vi kiến trúc Pipeline và Non-Pipeline, bài tập đã làm rõ một nghịch lý quan trọng trong thiết kế phần cứng: Việc áp dụng thanh ghi đường ống giúp tăng hiệu năng và triệt tiêu nhiễu chuyển mạch, nhưng sự ổn định đó lại vô tình làm cho tín hiệu rò rỉ trở nên "sạch" và sắc nét hơn, khiến hệ thống dễ bị trích xuất thông tin bí mật hơn so với kiến trúc đơn giản. Qua đó, bài tập khẳng định sự thiết yếu của việc tích hợp các kỹ thuật bảo vệ cho các lỗi tính toán trước khi triển khai vào thực tế.

Chương I. TỔNG QUAN.

1. Giới thiệu đề tài.

Mục tiêu của bài tập lớn là thiết kế một khối phần cứng RTL, thực hiện quy trình tổng hợp và mô phỏng mức vật lý nhằm thu thập dữ liệu chuyển mạch. Từ dữ liệu này, thực hiện đánh giá rò rỉ kênh bên bằng phương pháp kiểm định thống kê TVLA (Test Vector Leakage Assessment).

2. Bài toán thiết kế.

Khối phần cứng được lựa chọn để thiết kế là MAC Engine. Đây là khối tính toán cốt lõi trong các bộ lọc FIR hoặc mạng nơ-ron.

Lý do lựa chọn: Khối MAC có chứa bộ cộng dồn tuần tự và thực hiện phép nhân liên tục trong nhiều chu kỳ. Sự thay đổi trạng thái liên tục của thanh ghi cộng dồn tạo ra sự khác biệt rõ rệt về Switching Activity, rất lý tưởng để quan sát rò rỉ kênh bên.

Chương II. THIẾT KẾ RTL VÀ MÔ PHỎNG CHỨC NĂNG.

1. Kiến trúc RTL.

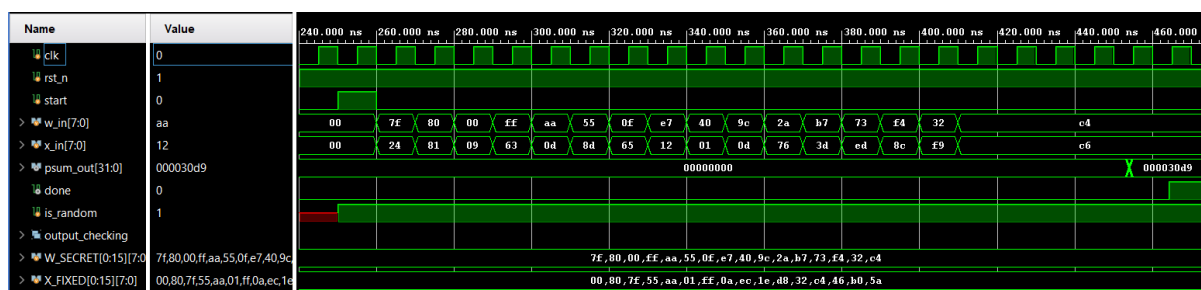
Thiết kế MAC Engine được viết bằng ngôn ngữ Verilog với các đặc điểm:

- Giao tiếp: Tín hiệu điều khiển cơ bản clk, rst_n, start, done.
- Dữ liệu: Đầu vào w_in (8-bit có dấu), x_in (8-bit có dấu); Đầu ra psum_out (32-bit có dấu).
- Hoạt động: Bộ điều khiển (FSM) bao gồm các trạng thái IDLE, CALC, DONE. Khi nhận lệnh start, mạch thực hiện tính toán $N=16$ cặp dữ liệu và tích lũy vào thanh ghi nội bộ p_reg.

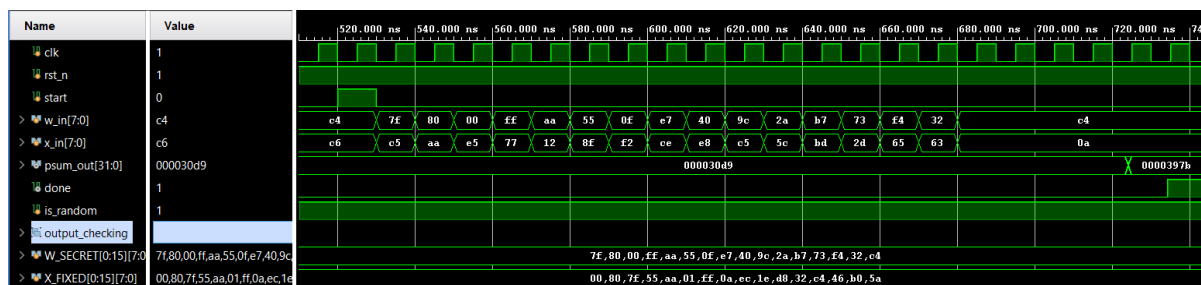
2. Phân tích chức năng.

Testbench được xây dựng theo cơ chế Self-checking. Trong mô phỏng Post-Implementation Timing, thiết kế đã khắc phục thành công độ trễ vật lý và tín hiệu GSR (Global Set/Reset) của FPGA.

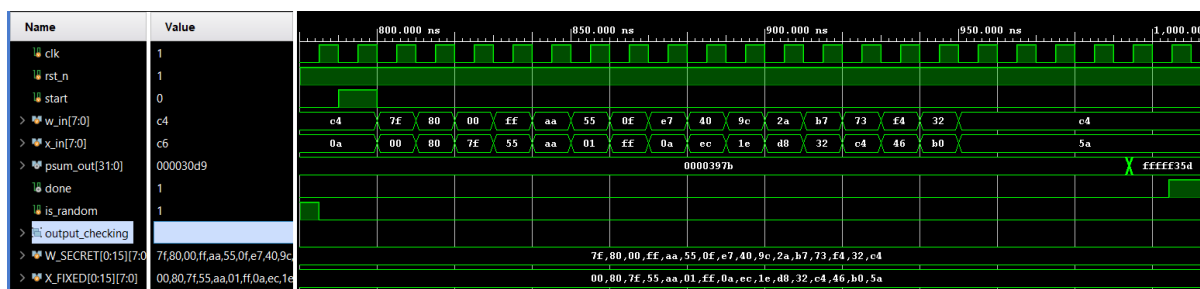
Kết quả: 100% các traces đều cho ra kết quả psum_out khớp tuyệt đối với mô hình toán học kỳ vọng, không xảy ra lỗi tràn số hay lệch pha FSM.



Hình 1: Waveform trace đầu tiên (x random)



Hình 2: Waveform trace tiếp theo (x random)



Hình 3: Waveform trace x fixed

Nhận xét:

- Về cơ chế cấp dữ liệu: Môi trường testbench kiểm soát luồng dữ liệu rất chính xác. Tùy thuộc vào trạng thái của cờ is_random, ngõ vào x_in sẽ được nạp tuần tự từ mảng hằng số X_FIXED (nếu cờ bằng 0) hoặc được cấp các giá trị ngẫu nhiên mới (nếu cờ bằng 1), đáp ứng đúng yêu cầu tạo tập mẫu cho phương pháp TVLA.

- Về đặc tính thời gian: Quan sát trên waveform, có thể thấy kết quả psum_out xuất hiện trễ 4 chu kỳ xung nhịp so với biến tham chiếu expected_psum. Sự chênh lệch này là hoàn toàn hợp lý vì: expected_psum hoạt động theo mô phỏng hành vi nên cho ra kết quả ngay lập tức. Trong khi đó, mạch phần cứng thực tế đòi hỏi dữ liệu phải dịch chuyển qua các trạng thái của khối điều khiển và các thanh ghi đường ống trong Datapath, tạo ra độ trễ tự nhiên là 5 chu kỳ.

- Về tính chính xác của phép toán: Tại thời điểm mạch bật cờ done báo hiệu kết thúc một trace tính toán, giá trị tích lũy cuối cùng tại ngõ ra psum_out khớp tuyệt đối 100% với giá trị kỳ vọng từ mô hình toán học trên toàn bộ 2000 mẫu mô phỏng.

Kết luận: Thiết kế MAC Engine đã hoàn toàn đáp ứng được yêu cầu về chức năng, kiểm soát luồng dữ liệu chuẩn xác và không xảy ra bất kỳ lỗi logic hay tràn số nào trong suốt quá trình hoạt động.

Chương III. TỔNG HỢP TÀI NGUYÊN.

Thiết kế được đưa qua quy trình Synthesis và Place-and-Route trên phần mềm Xilinx Vivado. Môi trường kiểm thử đặt tần số hoạt động ở mức tần số 100MHz.

Phân tích tài nguyên:

- LUT (Look-Up Tables): 45
- FF (Flip-Flops): 56
- DSP Blocks: 1
- Fmax: Mạch đáp ứng tốt timing constraint 100MHz (Slack dương: 6.847 ns).

Nhận xét:

- Về quy mô phần cứng: Thiết kế sử dụng tài nguyên cực kỳ nhỏ gọn với chỉ 45 LUTs và 56 FFs. Tỷ lệ FF lớn hơn LUT là hoàn toàn hợp lý đối với kiến trúc tuần tự, vì phần lớn tài nguyên được dùng cho thanh ghi tích lũy 32-bit, các thanh ghi đệm đầu vào/đầu ra và các Flip-Flop trạng thái của FSM. Điều này chứng tỏ RTL đã được viết rất tối ưu, không sinh ra các latch hay logic thừa.

- Về khối tính toán chuyên dụng: Thay vì dùng một lượng lớn LUT để tạo thành bộ nhân tổ hợp, phần mềm đã "map" thẳng phép nhân 8-bit vào lõi DSP48 chuyên dụng của Xilinx. Điều này không chỉ giúp tiết kiệm triệt để tài nguyên logic mà còn giúp mạch giảm tiêu thụ năng lượng rò rỉ và cải thiện mạnh mẽ tốc độ định tuyến.

- Về hiệu năng thời gian: Thiết kế vượt qua ràng buộc tần số 100MHz một cách cực kỳ dư dả với Slack dương lên tới 6.847 ns.

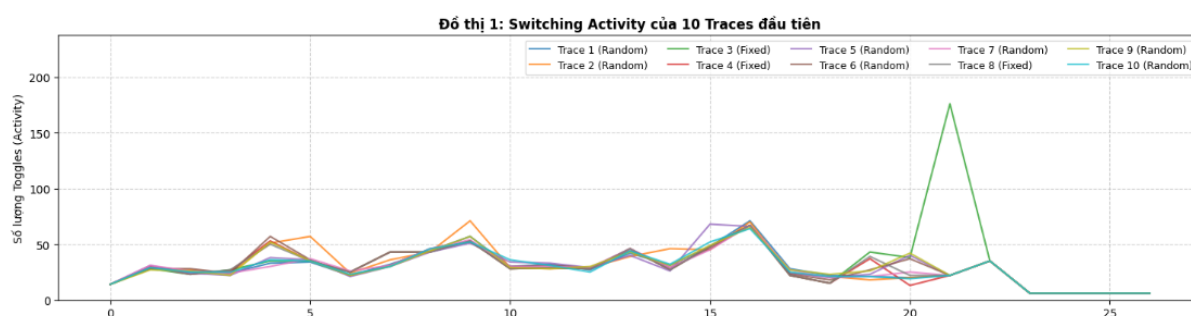
Chương IV. PHÂN TÍCH TVLA.

1. Môi trường thu thập traces cho TVLA.

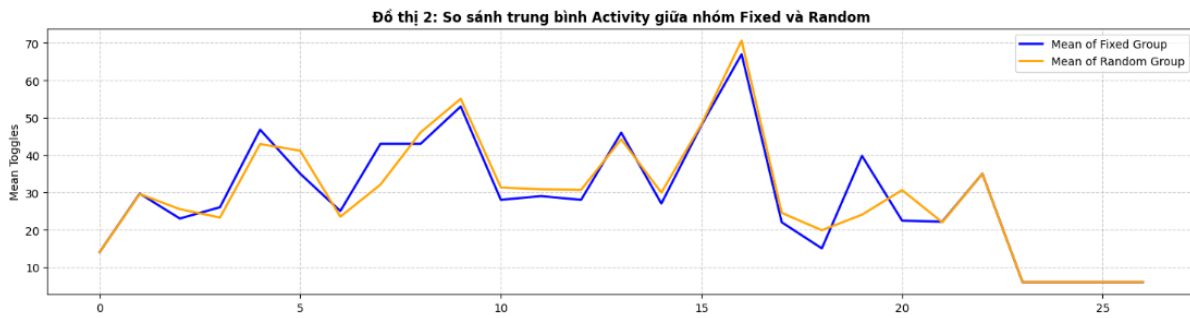
Để thực hiện phân tích TVLA nhằm đánh giá dấu vết hoạt động theo từng chu kỳ, môi trường mô phỏng được thiết lập như sau:

- Tập mẫu: Gồm 2 nhóm dữ liệu để thực hiện kiểm định *fixed* và *random*.
 - Nhóm 1 (Fixed): Khóa bí mật (W) được giữ cố định, đồng thời mảng dữ liệu đầu vào (X) cũng là một vector cố định xuyên suốt các lần chạy.
 - Nhóm 2 (Random): Khóa bí mật (W) giữ cố định, nhưng dữ liệu đầu vào (X) được sinh ngẫu nhiên ở mỗi chu kỳ.
- Số lượng mẫu: Chạy tổng cộng 2000 traces (phân bổ ngẫu nhiên 50-50 nhờ hàm \$urandom). Thực tế trích xuất được khỏi dữ liệu lý tưởng gồm 1000 traces Fixed và 1000 traces Random, hoàn toàn đáp ứng yêu cầu thống kê.
- Đại lượng thu thập: Quá trình mô phỏng sử dụng lệnh \$dumpvars để ghi lại waveform dưới dạng file mac_activity.vcd. Sau đó, một script Python được sử dụng để parse file VCD, chuẩn hóa dữ liệu thành các vector số theo thời gian bằng cách tính tổng số lần lật bit của toàn bộ các nút mạng nội bộ theo từng chu kỳ xung nhịp. Mỗi trace thu được kéo dài 27 chu kỳ clock.

2. Phân tích Switching Activity.



Hình 4: Đồ thị 1 - Switching Activity của 10 traces đại diện



Hình 5: Đồ thị 2 - Trung bình Toggle Count của nhóm Fixed và Random

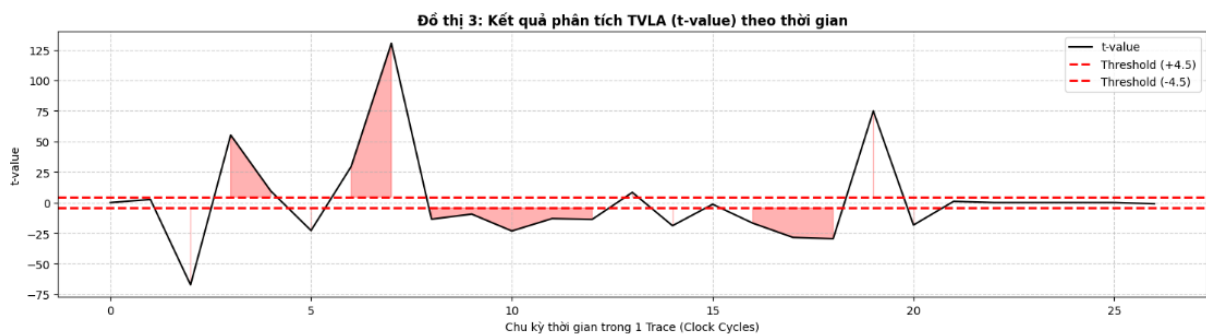
Nhận xét:

- Nhìn đồ thị 1: Hoạt động lật bit tạo thành một "vùng đồi" liên tục trong suốt pha tính toán CALC, kéo dài 23 chu kỳ (từ chu kỳ 0 đến 22). Có sự xuất hiện của đỉnh đột biến lên tới 176 toggles (ở chu kỳ 21) do tác động của các vector dữ liệu biên. Các traces còn lại dao động ổn định với đỉnh từ 64-71 toggles, tập trung quanh chu kỳ 16. Sau chu kỳ 22, mạch hoàn thành tính toán nên activity giảm mạnh về gần 0.

- Dựa vào đồ thị 2: Nhóm Random có mức độ chuyển mạch nhỉnh hơn nhóm Fixed. Cụ thể, đỉnh lật bit trung bình cao hơn (70.6 so với 67.0) và mức tiêu thụ trung bình trong toàn bộ pha CALC cũng lớn hơn (~33.7 so với ~33.4 toggles/chu kỳ).

Kết luận: Sự chênh lệch 0.3 toggles/chu kỳ cho thấy dữ liệu ngẫu nhiên làm gia tăng khoảng cách Hamming trung bình tại các cổng logic, khiến hệ thống phải lật bit thường xuyên và tiêu thụ nhiều năng lượng hơn so với tập dữ liệu cố định.

3. Phân tích Switching Activity.



Hình 6: Đồ thị 3 - Trị số t-value (Fixed-vs-Random) theo thời gian

Nhận xét:

Phương pháp Welch's T-test được áp dụng với ngưỡng tin cậy $C = \pm 4.5$.

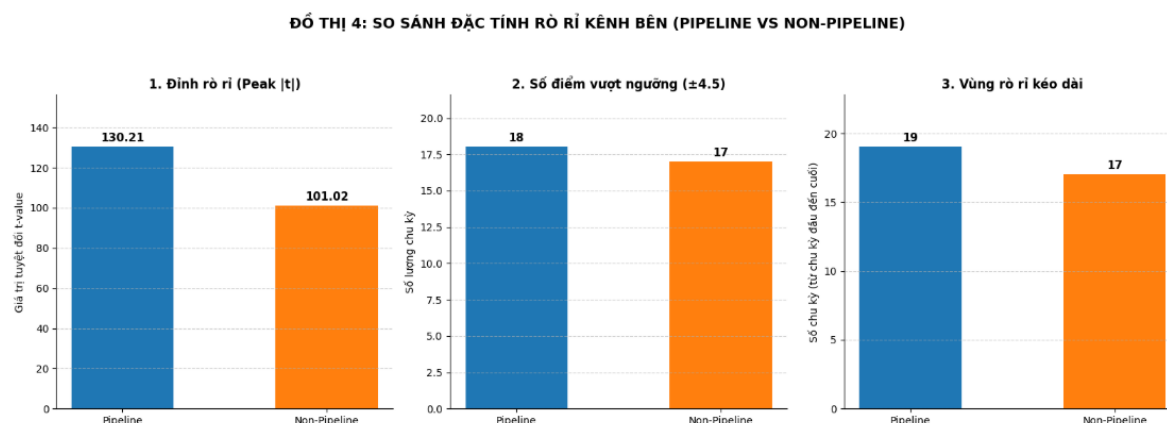
- Dựa vào Đồ thị 3, đường trị số t-value đã đâm thủng ngưỡng an toàn ± 4.5 một cách rõ rệt. Điều này khẳng định thiết kế MAC Engine hiện tại có rò rỉ thông tin kênh bên rất nghiêm trọng.

- Sự rò rỉ không rời rạc mà kéo dài trên 18 chu kỳ (tại các chu kỳ: 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20). Mức rò rỉ đạt đỉnh điểm (Peak $|t|$) lên tới 130.21 tại chu kỳ thứ 7.

- Toàn bộ 18 điểm vi phạm đều nằm trọn trong pha tính toán cốt lõi. Điều này chỉ ra bộ nhân và thanh ghi cộng dồn chính là các module gây lộ lọt thông tin, do trạng thái lật bit tại đây phụ thuộc trực tiếp vào khóa bí mật (W) và dữ liệu đầu vào (X).

4. So sánh kiến trúc Pipeline và Non-pipeline

Kiến trúc Non-Pipeline được sử dụng làm mốc đối chiếu nhằm hai mục đích: đánh giá sự đánh đổi giữa hiệu năng và bảo mật khi áp dụng kỹ thuật Pipeline; xác định nguyên nhân rò rỉ đến từ bản thân thuật toán hay do các thanh ghi khuếch đại.



Hình 7: So sánh trực quan 3 tiêu chí rò rỉ TVLA giữa Pipeline và Non-Pipeline.

Nhận xét: Dựa vào Đồ thị 4, tác động của vi kiến trúc lên mức độ rò rỉ được thể hiện qua 3 tiêu chí:

- Đỉnh rò rỉ (Peak $|t|$): Pipeline nghiêm trọng hơn rõ rệt (130.21 so với 101.02). Các thanh ghi của mạch Pipeline lưu trạng thái trung gian rất ổn định, tạo ra độ chênh lệch chuyển mạch sắc nét. Ngược lại, mạch Non-Pipeline sinh ra lượng nhiễu khổng lồ, vô tình tạo thành "ồn thuật toán" làm giảm độ lớn của đỉnh thống kê.

- Số điểm vượt ngưỡng (± 4.5): Tương đương nhau (18 chu kỳ ở Pipeline và 17 chu kỳ ở Non-Pipeline). Cả hai thiết kế đều thất bại. Việc chèn thêm tầng thanh ghi chỉ thay đổi hình dạng tín hiệu chứ không che giấu được sự rò rỉ.

- Vùng rò rỉ kéo dài: Pipeline kéo dài vùng rò rỉ lâu hơn (chu kỳ 2 đến 20) so với mạch Non-Pipeline (chu kỳ 2 đến 18). Đây là hệ quả tất yếu của độ trễ đường ống, khiến dữ liệu bí mật cần nhiều nhịp clock hơn để dịch chuyển hết qua các tầng tính toán.

Chương V. KẾT LUẬN.

Bài tập lớn đã hoàn thành toàn diện các mục tiêu đề ra, từ bước thiết kế RTL, tổng hợp tài nguyên phần cứng, cho đến việc mô phỏng và đánh giá mức độ rò rỉ bảo mật kênh bên cho khối tính toán MAC Engine. Qua các kết quả đạt được, báo cáo rút ra những kết luận trọng tâm sau:

- Về chức năng và hiệu năng phần cứng: Thiết kế kiến trúc Pipeline cho MAC Engine đã hoạt động chính xác 100% so với mô hình toán học kỳ vọng, kiểm soát tốt độ trễ và luồng dữ liệu. Đặc biệt, quá trình tổng hợp trên Xilinx Vivado cho thấy thiết kế cực kỳ tối ưu về mặt tài nguyên, đồng thời vượt qua ràng buộc thời gian 100MHz một cách dễ dàng với lượng Slack dương lớn.

- Về đánh giá bảo mật: Mặc dù hoàn hảo về mặt logic và hiệu năng, thiết kế lại bộc lộ điểm yếu chí mạng về bảo mật phần cứng. Kết quả kiểm định Welch's T-test đã phá vỡ hoàn toàn ngưỡng an toàn ± 4.5 , chứng minh rằng tiêu thụ năng lượng của mạch có sự phụ thuộc trực tiếp và rõ rệt vào tập dữ liệu đầu vào. Toàn bộ pha tính toán cốt lõi đều bị rò rỉ thông tin.

- Về nghịch lý vi kiến trúc: Phân tích đối chiếu đã chỉ ra một hệ quả đáng chú ý trong thiết kế phần cứng: Việc áp dụng kỹ thuật Pipeline giúp chặn đứng nhiễu chuyên mạch và tối ưu hóa năng lượng/tốc độ, nhưng sự ổn định của các thanh ghi lại vô tình tạo ra một tín hiệu rò rỉ "sạch" và ít nhiễu. Điều này khiến kiến trúc Pipeline dễ bị tấn công trích xuất thông tin bí mật hơn cả kiến trúc Non-Pipeline.

PHỤ LỤC

1. Code RTL và testbench và dữ liệu Trace: [GitHub](#)
2. Script python phân tích TVLA: [Kaggle](#)
3. Slide báo cáo: [Slide](#)